ARCANE ASSEMBLY

Justificación del Esquema Entidad-Relación

Juego	Arcane Assembly
Curso	TC2005B — Construcción de Software y Toma de Decisiones
Versión ER	v2 (marzo 2026)
Entidades	8
Relaciones	9

1. Visión General del Esquema

Arcane Assembly es un juego roguelite donde el jugador construye un tablero hexagonal de 5×5 antes de cada combate, elige una ruta en un mapa de nodos procedural, y acumula piezas a través de un sistema de draft. La base de datos registra la sesión completa de un jugador desde que inicia una run hasta que gana o pierde, pasando por cada nodo del mapa, cada combate y cada pieza colocada en el tablero.

El esquema v2 amplía el diseño anterior añadiendo la entidad JUGADOR (para soporte multijugador futuro), separando EVENTO_NODO de NODO para mantener la normalización, y ajustando los atributos de RUN para reflejar los límites de uso implementados en el prototipo HTML (máx. 2 visitas a tienda, máx. 3 eventos por run).

Resumen de entidades

Entidad	Descripción
JUGADOR	Perfil persistente del jugador
RUN	Una partida completa (roguelite)
NODO	Casilla del mapa: combate, tienda, evento…
ENEMIGO	Catálogo de enemigos del juego
COMBATE	Registro de un enfrentamiento en un nodo
COLOCACION_TABLERO	Pieza colocada en el tablero en un combate
PIEZA	Catálogo de piezas disponibles
INVENTARIO_RUN	Piezas que posee el jugador en una run
EVENTO_NODO	Efecto disparado al entrar a un nodo evento

2. Entidades, Atributos y Dominios

2.1 JUGADOR

Almacena el perfil de cada jugador. Permite rastrear estadísticas globales como el total de runs realizadas y el tiempo acumulado de juego, facilitando un sistema de logros o rankings futuro.

Atributo	Dominio / Tipo	Restricción	Justificación
jugador_id	INT	PK, AUTO_INCREMENT	Clave sintética; evita dependencia de nombre o email
nombre	VARCHAR(60)	NOT NULL	Apodo visible en interfaz
email	VARCHAR(120)	UNIQUE, NULLABLE	Identificación opcional para login futuro
fecha_registro	DATETIME	DEFAULT NOW()	Trazabilidad de cuenta
total_runs	INT	DEFAULT 0	Contador desnormalizado para rendimiento
tiempo_total_seg	INT	DEFAULT 0	Suma de duración de todas las runs

2.2 RUN

Representa una partida completa. Modela el estado actual del jugador durante la run: su HP, sus recursos de impulso, y el contador de usos de tienda/evento que el prototipo limita por diseño.

Atributo	Dominio / Tipo	Restricción	Justificación
run_id	INT	PK, AUTO_INCREMENT	Identificador único de partida
jugador_id	INT	FK → JUGADOR, NOT NULL	Asocia la run a su propietario
fecha_inicio	DATETIME	NOT NULL	Permite calcular duración total
zona_actual	INT	CHECK (1..3), DEFAULT 1	El mapa tiene 3 zonas; CHECK previene corrupción
hp_actual	INT	NOT NULL	Vida restante; derrota si llega a 0
impulso	INT	DEFAULT 3	Recurso escaso para activar piezas fuera de orden
usos_tienda	INT	DEFAULT 0	Máx 2 por run (regla de diseño)
usos_evento	INT	DEFAULT 0	Máx 3 por run (regla de diseño)
resultado	ENUM(victoria,derrota,en_progreso)	NOT NULL	Estado terminal o activo de la run

2.3 PIEZA

Catálogo estático de todas las piezas del juego. Los atributos numéricos son NULLABLE porque no todos los tipos los usan (ej. un Ancla no tiene output_base).

Atributo	Dominio / Tipo	Restricción	Justificación
pieza_id	VARCHAR(20)	PK, NOT NULL	ID semántico: 'gen_energia', 'cat_amplif', etc.
nombre	VARCHAR(60)	NOT NULL	Nombre mostrado en UI
tipo	ENUM(generator,transformer,catalyst,anchor)	NOT NULL	Los 4 arquetipos del GDD
rareza	INT	CHECK (0..4), NOT NULL	0=común … 4=legendario; afecta probabilidad de draft
output_base	DECIMAL	NULLABLE	Solo Generadores y Transformadores lo tienen
multiplicador	DECIMAL	NULLABLE	Solo Transformadores
amplificacion	DECIMAL	NULLABLE	Solo Catalizadores (+25% a +80%)
escudo_val	INT	NULLABLE	Solo Anclas con efecto escudo
descripcion	TEXT	NOT NULL	Texto de la carta mostrado en UI

2.4 NODO

Un nodo es una casilla del mapa generado proceduralmente. Cada run tiene entre 15 y 20 nodos distribuidos en 3 zonas. El atributo tipo determina qué ocurre al seleccionarlo.

Atributo	Dominio / Tipo	Restricción	Justificación
nodo_id	INT	PK, AUTO_INCREMENT	Clave sintética
run_id	INT	FK → RUN, NOT NULL	Pertenece a una sola run
tipo	ENUM(start,combat,elite,boss,shop,event)	NOT NULL	Determina la lógica al visitar el nodo
zona	INT	CHECK (1..3), NOT NULL	Zona 3 = boss; controla escalado de dificultad
completado	BOOLEAN	DEFAULT FALSE	Permite marcar nodos ya visitados
accesible	BOOLEAN	DEFAULT FALSE	Solo los nodos desbloqueados por el camino
enemigo_id	INT	FK → ENEMIGO, NULLABLE	NULL en nodos de tipo shop, event, start
fila_mapa/col_mapa	INT	NOT NULL	Posición en el grid visual del mapa

2.5 ENEMIGO

Catálogo de enemigos. El campo build modela el estilo de tablero que construye el enemigo automáticamente: aggro prioriza daño, tanky usa anclas de escudo, amplify apila catalizadores, mixed mezcla tipos, y boss tiene una configuración única predefinida.

Atributo	Dominio / Tipo	Restricción	Justificación
enemigo_id	INT	PK, AUTO_INCREMENT	Clave sintética
nombre	VARCHAR(60)	NOT NULL	Nombre mostrado en pantalla de combate
build	ENUM(aggro,tanky,amplify,mixed,boss)	NOT NULL	Determina el algoritmo de colocación enemiga
hp_base	INT	NOT NULL	HP inicial antes de escalar por zona
flavor_text	TEXT	NULLABLE	Texto narrativo opcional en pantalla de combate
tipo_nodo	ENUM(combat,elite,boss)	NOT NULL	Restringe en qué nodos puede aparecer

2.6 COMBATE

Registra la ejecución de un combate. La relación con NODO es 1:1 reforzada con UNIQUE en nodo_id. El campo sinergias_activadas es un contador desnormalizado para analíticas.

Atributo	Dominio / Tipo	Restricción	Justificación
combate_id	INT	PK, AUTO_INCREMENT	Clave sintética
nodo_id	INT	FK → NODO, UNIQUE, NOT NULL	UNIQUE impone relación 1:1 con NODO
ronda_actual	INT	DEFAULT 1	Número de ronda de resolución actual
hp_enemigo	INT	NOT NULL	HP actual del enemigo durante el combate
sinergias_activadas	INT	DEFAULT 0	Contador de sinergias disparadas en la ronda
resultado	ENUM(victoria,derrota)	NULLABLE	NULL mientras el combate está en curso
dano_total_infligido	INT	DEFAULT 0	Analítica: daño acumulado del jugador

2.7 COLOCACION_TABLERO

Registra cada pieza colocada en el tablero (del jugador o del enemigo) durante un combate. La posición se almacena en coordenadas de columna/fila del grid hexagonal 5×5.

Atributo	Dominio / Tipo	Restricción	Justificación
colocacion_id	INT	PK, AUTO_INCREMENT	Clave sintética
combate_id	INT	FK → COMBATE, NOT NULL	Pertenece a un combate
pieza_id	VARCHAR(20)	FK → PIEZA, NOT NULL	Referencia al catálogo
col_hex / fila_hex	INT	CHECK (0..4), NOT NULL	Grid 5×5; CHECK previene posiciones fuera de rango
propietario	ENUM(jugador, enemigo)	NOT NULL	Distingue tableros del jugador y del enemigo
ronda	INT	NOT NULL	Ronda en la que se colocó la pieza

2.8 INVENTARIO_RUN

Tabla de intersección N:M entre RUN y PIEZA. Representa el mazo del jugador durante una partida. Permite tener varias copias de la misma pieza mediante el campo cantidad.

Atributo	Dominio / Tipo	Restricción	Justificación
run_id	INT	FK → RUN, NOT NULL	Parte de la clave compuesta
pieza_id	VARCHAR(20)	FK → PIEZA, NOT NULL	Parte de la clave compuesta
cantidad	INT	DEFAULT 1, CHECK (>=1)	Permite varias copias de la misma pieza
origen	ENUM(inicial,draft,evento,tienda)	NOT NULL	Trazabilidad de cómo se obtuvo la pieza
nodo_adquirido_id	INT	FK → NODO, NULLABLE	Nodo donde se obtuvo; NULL si es pieza inicial

2.9 EVENTO_NODO

Almacena el efecto de un nodo de tipo event. Se separó de NODO para mantener la tercera forma normal: los atributos tipo_efecto, valor_efecto y eleccion_jugador solo aplican a nodos de tipo event.

Atributo	Dominio / Tipo	Restricción	Justificación
evento_id	INT	PK, AUTO_INCREMENT	Clave sintética
nodo_id	INT	FK → NODO, NOT NULL	Un evento pertenece a un nodo
titulo	VARCHAR(60)	NOT NULL	Nombre del evento mostrado en pantalla
tipo_efecto	ENUM(draft,heal,damage,impulse,skip)	NOT NULL	Determina el efecto sobre el estado del jugador
valor_efecto	INT	NULLABLE	Cuánto HP, impulso o daño aplica; NULL si tipo=draft
eleccion_jugador	VARCHAR(60)	NULLABLE	Opción elegida en eventos con elección

3. Relaciones y Cardinalidades

La siguiente tabla resume las 9 relaciones del esquema con sus cardinalidades y la clave foránea que las implementa.

Relación	Card.	FK en…	Notas de integridad
JUGADOR → RUN	1:N	RUN.jugador_id	ON DELETE RESTRICT: no borrar jugador con runs activas
RUN → NODO	1:N	NODO.run_id	ON DELETE CASCADE: borrar run elimina todos sus nodos
NODO → ENEMIGO	N:0..1	NODO.enemigo_id NULLABLE	NULL en nodos shop/event/start; FK NULLABLE
NODO → COMBATE	1:1	COMBATE.nodo_id UNIQUE	UNIQUE garantiza exactamente 1 combate por nodo
COMBATE → COLOCACION	1:N	COLOCACION.combate_id	ON DELETE CASCADE: borrar combate limpia tablero
PIEZA → COLOCACION	1:N	COLOCACION.pieza_id	ON DELETE RESTRICT: no borrar pieza usada en tablero
RUN → INVENTARIO_RUN	1:N	INVENTARIO_RUN.run_id	ON DELETE CASCADE: fin de run borra inventario
PIEZA → INVENTARIO_RUN	1:N	INVENTARIO_RUN.pieza_id	ON DELETE RESTRICT: catálogo es inmutable
NODO → EVENTO_NODO	1:0..1	EVENTO_NODO.nodo_id	Solo nodos de tipo=event tienen un registro aquí

Decisiones de cardinalidad destacadas

- NODO ↔ COMBATE es 1:1: Cada nodo de tipo combat, elite o boss genera exactamente un combate cuando el jugador lo visita. La restricción UNIQUE en COMBATE.nodo_id impone esta cardinalidad a nivel de base de datos.
- NODO → ENEMIGO es N:0..1: El mismo enemigo puede aparecer en múltiples nodos. La cardinalidad 0..1 del lado ENEMIGO refleja que los nodos shop, event y start no tienen enemigo asignado.
- RUN ↔ PIEZA a través de INVENTARIO_RUN: Se usa una tabla de intersección porque el inventario tiene atributos propios (cantidad, origen) y necesita estar normalizado para consultas eficientes.

4. Restricciones de Integridad

4.1 Restricciones de dominio (CHECK)

Tabla.Atributo	Restricción	Motivo de diseño
RUN.zona_actual	CHECK (1 <= zona_actual <= 3)	El mapa tiene exactamente 3 zonas
NODO.zona	CHECK (1 <= zona <= 3)	Consistente con la estructura del mapa
PIEZA.rareza	CHECK (0 <= rareza <= 4)	5 niveles de rareza (0=común, 4=legendario)
COLOCACION.col_hex	CHECK (0 <= col_hex <= 4)	Grid hexagonal de 5 columnas
COLOCACION.fila_hex	CHECK (0 <= fila_hex <= 4)	Grid hexagonal de 5 filas
INVENTARIO_RUN.cantidad	CHECK (cantidad >= 1)	No puede haber 0 copias de una pieza en inventario
RUN.resultado	ENUM(victoria,derrota,en_progreso)	Solo 3 estados terminales posibles
COMBATE.resultado	ENUM(victoria,derrota) NULLABLE	NULL mientras el combate está activo

4.2 Restricciones de unicidad

Tabla	Constraint UNIQUE	Efecto
JUGADOR	UNIQUE (email)	Un email solo puede pertenecer a un jugador
COMBATE	UNIQUE (nodo_id)	Impone la relación 1:1 NODO ↔ COMBATE
COLOCACION_TABLERO	UNIQUE (combate_id, col_hex, fila_hex, propietario, ronda)	Impide dos piezas en la misma celda del mismo tablero en la misma ronda

4.3 Acciones referenciales (ON DELETE / ON UPDATE)

Clave Foránea	ON DELETE	ON UPDATE
RUN.jugador_id → JUGADOR	RESTRICT	CASCADE
NODO.run_id → RUN	CASCADE	CASCADE
NODO.enemigo_id → ENEMIGO	SET NULL	CASCADE
COMBATE.nodo_id → NODO	CASCADE	CASCADE
COLOCACION.combate_id → COMBATE	CASCADE	CASCADE
COLOCACION.pieza_id → PIEZA	RESTRICT	CASCADE
INVENTARIO_RUN.run_id → RUN	CASCADE	CASCADE
INVENTARIO_RUN.pieza_id → PIEZA	RESTRICT	CASCADE
EVENTO_NODO.nodo_id → NODO	CASCADE	CASCADE

5. Normalización

El esquema cumple con la Tercera Forma Normal (3NF):

1NF — Atomicidad

Todos los atributos contienen valores atómicos. Los ENUMs representan dominios discretos finitos. No existen grupos repetitivos ni atributos multivaluados (las piezas del inventario se modelan como filas en INVENTARIO_RUN, no como un array JSON).

2NF — Sin dependencias parciales

Todas las claves primarias son simples (INT AUTO_INCREMENT o VARCHAR), excepto INVENTARIO_RUN cuya clave compuesta (run_id, pieza_id) determina completamente todos sus atributos no clave. No hay atributos que dependan solo de parte de una clave compuesta.

3NF — Sin dependencias transitivas

No existen atributos que dependan de otros atributos no clave. Los datos del enemigo (hp_base, build) residen en ENEMIGO, no en NODO. Los atributos de evento (tipo_efecto, valor_efecto) se separaron en EVENTO_NODO precisamente para eliminar la dependencia transitiva que habrían causado en NODO.

6. Trazabilidad con el GDD

La siguiente tabla mapea cada mecánica descrita en el GDD con la entidad o atributo que la soporta en el esquema.

Mecánica del GDD	Entidad / Atributo	Notas
Tablero hexagonal 5×5	COLOCACION_TABLERO: col_hex, fila_hex	CHECK 0..4 en ambos
4 tipos de pieza	PIEZA.tipo ENUM	generator, transformer, catalyst, anchor
5 piezas por ronda (draft)	INVENTARIO_RUN.origen = 'draft'	Lógica de límite en capa de aplicación
Sistema de Impulso	RUN.impulso INT	Decrementa al usar; se recupera entre nodos
Mapa de nodos procedural	NODO: tipo, zona, fila_mapa, col_mapa	Generado al crear la RUN
Tienda (máx 2 usos)	RUN.usos_tienda INT	Incrementado al visitar nodo shop
Eventos (máx 3 usos)	RUN.usos_evento + EVENTO_NODO	Registro y límite de eventos
Escalado por zona	NODO.zona + ENEMIGO.hp_base	hp_real = hp_base * zona (en aplicación)
Boss final	ENEMIGO.build = 'boss'	Tablero único y predefinido
Sinergias entre piezas	COMBATE.sinergias_activadas	Calculadas en resolución automática
Draft de recompensas	INVENTARIO_RUN.origen = 'draft'	Piezas elegidas tras ganar combate
Rareza de piezas	PIEZA.rareza CHECK(0..4)	Controla probabilidad en draft
Run con victoria/derrota	RUN.resultado ENUM	Actualizado al finalizar la partida

7. Consultas Representativas

Ejemplos de consultas SQL que validan que el esquema soporta las operaciones requeridas por el juego:

7.1 Inventario completo de una run activa

```
SELECT p.nombre, p.tipo, p.rareza, ir.cantidad, ir.origen
FROM INVENTARIO_RUN ir
JOIN PIEZA p ON p.pieza_id = ir.pieza_id
WHERE ir.run_id = :run_id
ORDER BY p.rareza DESC, p.tipo;
```

7.2 Reconstruir el tablero del jugador en un combate

```
SELECT p.nombre, p.tipo, ct.col_hex, ct.fila_hex, ct.ronda
FROM COLOCACION_TABLERO ct
JOIN PIEZA p ON p.pieza_id = ct.pieza_id
WHERE ct.combate_id = :combate_id AND ct.propietario = 'jugador'
ORDER BY ct.ronda, ct.fila_hex, ct.col_hex;
```

7.3 Historial de runs del jugador

```
SELECT r.run_id, r.fecha_inicio, r.zona_actual, r.hp_actual, r.resultado, COUNT(n.nodo_id) AS nodos_completados
FROM RUN r
LEFT JOIN NODO n ON n.run_id = r.run_id AND n.completado = TRUE
WHERE r.jugador_id = :jugador_id
GROUP BY r.run_id
ORDER BY r.fecha_inicio DESC;
```

7.4 Estadísticas de sinergias por tipo de pieza

```
SELECT p.tipo, AVG(c.sinergias_activadas) AS avg_sinergias, COUNT(*) AS combates
FROM COMBATE c
JOIN COLOCACION_TABLERO ct ON ct.combate_id = c.combate_id
JOIN PIEZA p ON p.pieza_id = ct.pieza_id
WHERE ct.propietario = 'jugador'
GROUP BY p.tipo
ORDER BY avg_sinergias DESC;
```